SpaceWases: cómo funciona por dentro

Andrés Felipe Verbel Pacheco
Universidad de la Costa (CUC), Barranquilla, Colombia · averbel1@cuc.edu.co · 1 de noviembre de 2026

SpaceWases envía texto entre dos dispositivos usando solo sonido: uno convierte el mensaje en tonos audibles y el otro los escucha con el micrófono y los vuelve a convertir en texto. Este documento explica cómo lo hace la web publicada en https://spacewases.vercel.app, desde que alguien pulsa Enviar hasta que el mensaje aparece en el otro dispositivo. Describe la versión publicada el 1 de noviembre de 2026, con los valores que usa el código.

1. Visión general

La aplicación es una página estática: no tiene servidor ni base de datos, y los mensajes no pasan por internet. Vercel solo entrega los archivos; después, todo ocurre en el navegador de cada dispositivo. Los mensajes viajan únicamente por el aire de la sala.

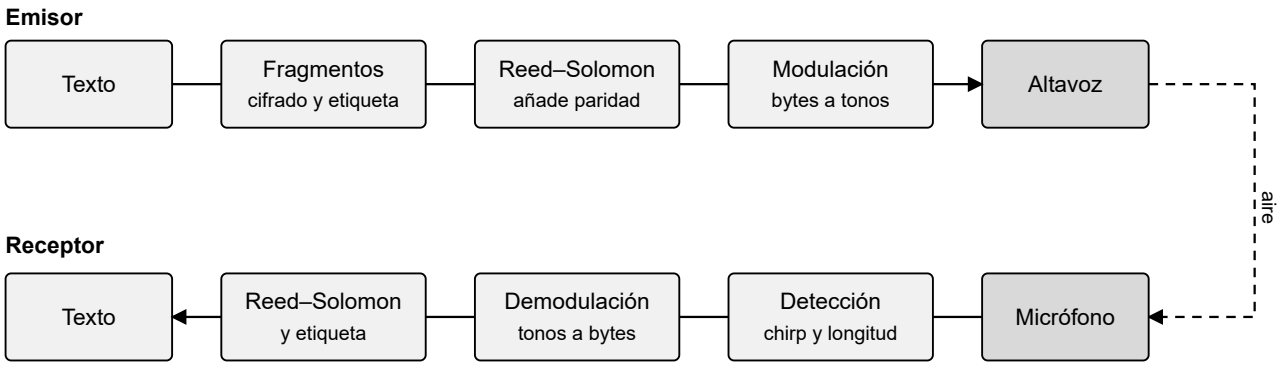


Fig. 1. Recorrido de un mensaje. Cada paso del emisor tiene su paso inverso en el receptor.

Un byte se transmite como un sonido de dos tonos simultáneos. Como el aire de una sala introduce ruido y eco, el diseño añade tres defensas: símbolos largos, para que el eco no contamine el siguiente; Reed–Solomon, para corregir los bytes dañados; y unos barridos de frecuencia llamados *chirps*, para saber dónde empieza el mensaje.

Los archivos de la web

Archivo	Responsabilidad
`index.html`, `style.css`	Estructura y aspecto: pestañas Ajustes, Mensajes y Espectro, con la disposición de Waver.
`app.js`	Interfaz: botones, historial, reproducción del sonido, permiso del micrófono, espectro y coordinación de los demás hilos.
`spacewases.js`	El módem: Reed–Solomon, tramas, cifrado, modulación y recepción. No toca la página, por eso corre también en Node para las pruebas.
`captura.js`	Corre en el hilo de audio: agrupa el sonido del micrófono en bloques de 4096 muestras.
`receptor.js`	Corre en un hilo aparte (`Worker`): recibe el audio y ejecuta la recepción sin frenar la página.

La carpeta `SpacesWases/spacewases/` contiene la misma lógica de módem en Python, con el mismo formato de señal, para las pruebas por aire, el banco de ruido y la línea de comandos. Un mensaje generado en uno se decodifica en el otro.

2. Envío: del texto a los bytes

Al pulsar Enviar, `app.js` convierte el texto a UTF-8 y llama a `send()`. Los bytes se empaquetan así:

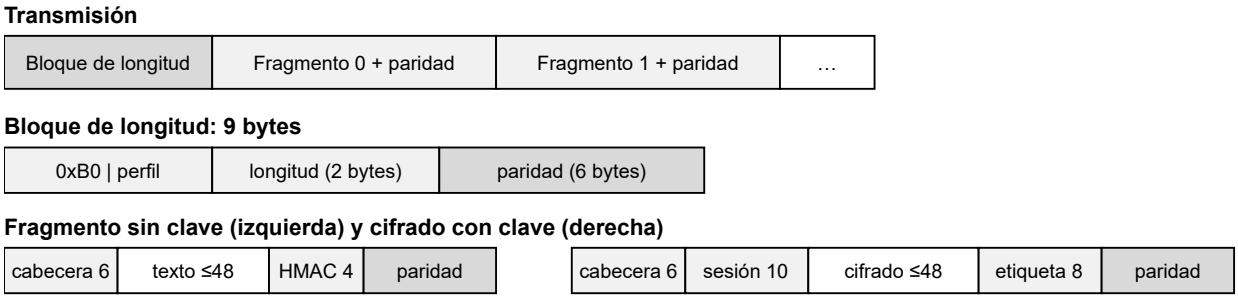


Fig. 2. Bytes de una transmisión. Los de paridad corrigen errores; el HMAC o la etiqueta comprueban que el contenido es correcto.

- **Fragmentos.** El texto se parte en trozos de hasta 48 bytes, sin relleno. El límite es de 255 fragmentos, es decir, 12 240 bytes; la caja de texto admite 2000 caracteres.
- **Cabecera.** Seis bytes: marca fija `0xA5`, versión y perfil, identificador del mensaje, número de fragmento, total de fragmentos y longitud del trozo.
- **Bloque de longitud.** Va primero y dice el perfil y la longitud total. Tiene su propia paridad: corrige hasta 3 de sus 9 bytes. Permite calcular de antemano dónde acaba el mensaje, y su validez distingue un mensaje real de una falsa detección.
- **Reed–Solomon.** Trabaja sobre GF(256) con el polinomio `0x11D` y añade paridad a cada fragmento. Con *r* bytes de paridad corrige hasta *r*/2 bytes erróneos, estén donde estén, y falla si hay más.
- **Repeticiones.** Si en Ajustes se eligen 2 o 3, se envía la secuencia completa de nuevo. Los fragmentos buenos de cualquier copia completan el mensaje.

Los tres protocolos

Protocolo	Duración del símbolo	Tramas sumadas	Paridad	Corrige	16 B	Tasa con 16 B
Robusto (por defecto)	128 ms	5 de 6	20 B	10 B	7,36 s	17,4 bps
Normal	64 ms	2 de 3	32 B	16 B	4,60 s	27,8 bps
Rápido	64 ms	2 de 3	20 B	10 B	3,84 s	33,4 bps

Ejemplo. Enviar 16 bytes sin clave con Robusto da un fragmento de 6 + 16 + 4 = 26 bytes más 20 de paridad, es decir, 46. Con los 9 del bloque de longitud son 55 bytes. A 128 ms cada uno suman 7,04 s, y los chirps y las guardas añaden 0,32 s: 7,36 s en total. Con clave, cabecera y etiqueta añaden 14 bytes: 69 bytes y 9,15 s.

3. Modulación: de los bytes al sonido

El audio se genera a 48 000 muestras por segundo. La señal completa tiene esta forma:

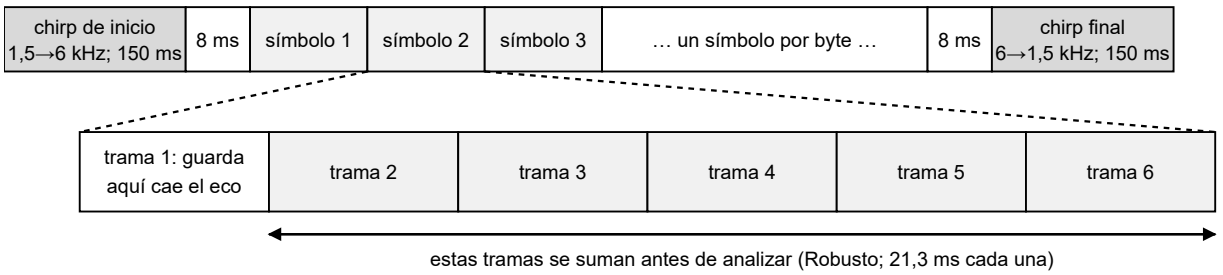


Fig. 3. Estructura de la señal y detalle de un símbolo de Robusto. En Normal y Rápido cada símbolo tiene 3 tramas.

Tonos. Cada trama dura 1024 muestras, así que la FFT separa frecuencias múltiplo de 46,875 Hz, llamadas *bins*. Hay dos grupos de 16 tonos, separados entre sí 6 bins: el grupo A usa los bins $40 + 6k$ (de 1875 a 6094 Hz) y el grupo B los bins $43 + 6k$ (de 2016 a 6234 Hz). Cada byte se divide en dos mitades de 4 bits, o *nibbles*: la alta elige un tono del grupo A y la baja, uno del grupo B. Ambos suenan a la vez durante todo el símbolo.

Por qué símbolos largos. En una sala el sonido sigue rebotando un momento después de emitirse. La primera trama recibe ese eco del símbolo anterior y se descarta; las demás llevan solo el tono actual y se suman, lo que refuerza el tono frente al ruido. Es la idea de ggwave, tomada como referencia, y es lo que permitió pasar por aire de 9 a 142 mensajes de 180. Unos bordes suaves de 64 muestras evitan chasquidos entre símbolos.

Reproducción. La señal se multiplica por el volumen de Ajustes y `app.js` la reproduce con Web Audio. Mientras emite, descarta lo que capta el micrófono para no recibir su propia señal: un dispositivo no escucha mientras habla.

4. Recepción: tres hilos

Al pulsar «Activar micrófono», la página pide permiso y solicita desactivar la cancelación de eco, la supresión de ruido y el control de ganancia, porque dañarían los tonos. Algunos navegadores pueden no respetarlo. El trabajo se reparte en tres hilos, para que uno ocupado no perjudique a los otros:

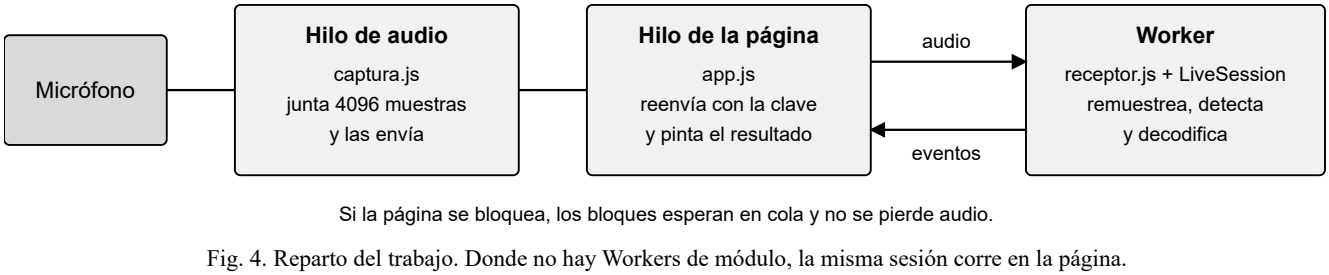


Fig. 4. Reparto del trabajo. Donde no hay Workers de módulo, la misma sesión corre en la página.

1. **Captura.** Un `AudioWorklet` corre en el hilo de audio y entrega bloques de 4096 muestras (85 ms). Donde no existe, se usa `ScriptProcessor`, que puede perder audio si la página se bloquea.
2. **Remuestreo.** Se pide audio a 48 kHz; si el navegador no lo admite con ese micrófono, se usa su frecuencia y `Resampler` la convierte a 48 kHz, conservando la posición entre trozos.
3. **Cola.** `LiveSession` procesa el audio en orden. Lo que llega mientras decodifica se junta y se procesa después, así que el receptor admite trozos de cualquier tamaño.

Detección del mensaje

El receptor compara continuamente el audio con la forma del chirp mediante una correlación normalizada, con FFT por bloques de 65 536 puntos como máximo. El resultado es una puntuación de 0 a 1 que no mide el volumen, sino cuánto destaca el chirp frente al resto del sonido. Usa dos umbrales distintos:

- **Inicio, 0,07.** Es bajo a propósito. En las pruebas, el ambiente de la sala no pasó de 0,042 y los sonidos sintéticos, de 0,075. Una falsa detección casi no cuesta, porque la descarta el paso siguiente.
- **Fin, 0,25.** Es alto porque los tonos de los datos llegan a unos 0,14, y un fin falso cortaría el mensaje.

Cuando la puntuación supera el umbral, el receptor espera a tener el chirp entero y toma el máximo, porque un chirp a medio llegar ya da una posición adelantada. Después avanza por estos estados:

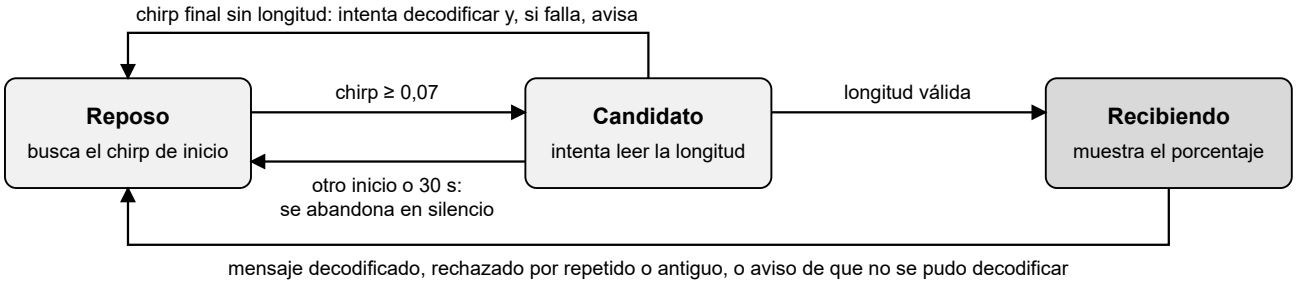


Fig. 5. Estados de `StreamReceiver`. El estado Candidato no se muestra en la interfaz.

- **Candidato.** Prueba los tres protocolos sobre los 9 bytes del bloque de longitud y cuatro posiciones de ventana cada uno. Una coincidencia al azar es muy improbable, porque debe pasar Reed–Solomon y además traer la marca exacta del protocolo.
- **Recibiendo.** Con la longitud calcula dónde acaba cada copia y decodifica al llegar ahí, sin esperar el chirp final. Si falla, espera otra copia, hasta 3, por si el mensaje venía repetido. Si aparece el chirp final antes, hace un último intento.
- **Tope.** Ninguna grabación supera 90 s.

Demodulación y validación

1. **Tonos a bytes.** En cada símbolo se suman las tramas sin la de guarda, se quita la media y se calcula la FFT. En cada grupo gana el tono con más potencia, sumando su bin y los dos vecinos. Los dos ganadores forman el byte.
2. **Alineación.** Si no decodifica, se repite con la ventana desplazada −256, −512 y +256 muestras. Si un fin de datos falso cortó el mensaje, se repite sin él.
3. **Corrección.** Cada bloque pasa por Reed–Solomon por separado. Un bloque dañado no descarta a los demás.
4. **Validación.** Se comprueban la marca, la versión, el protocolo y la etiqueta. Con clave, se descifra después de comprobarla.
5. **Reensamblado.** Los fragmentos se unen por identificador y sesión. El mensaje solo se entrega cuando están todos.
6. **Interfaz.** El Worker envía el resultado a la página, que lo añade al historial como `hora | Recv | protocolo | texto`, siempre como texto y nunca como HTML.

5. Clave y seguridad

La clave es opcional. Sin ella, el mensaje viaja en claro y la etiqueta de 4 bytes usa una clave vacía: detecta errores, pero cualquiera puede fabricar un mensaje. Con clave ocurre lo siguiente:

Paso	Cómo se hace
Clave maestra	PBKDF2-HMAC-SHA256 de la frase, con 600 000 iteraciones y sal fija `spacewases-pbkdf2-v1`. Hace costoso probar frases, pero no salva una frase débil.
Subclaves	HMAC de la clave maestra con `spacewases-v2-enc` y `spacewases-v2-mac`: una para cifrar y otra para autenticar.
Sesión	10 bytes por mensaje: marca de tiempo de 4 bytes y 6 aleatorios de `crypto.getRandomValues`.
Cifrado	XOR con un flujo HMAC-SHA256 en modo contador, sobre sesión, identificador, fragmento y contador.
Etiqueta	HMAC de cabecera, sesión y texto cifrado, truncado a 8 bytes. Se comprueba en tiempo constante antes de descifrar. Como cubre también el total de fragmentos, no se pueden quitar trozos.
Repeticiones	`ReplayGuard` rechaza sesiones ya vistas o fuera de una ventana de 120 s, para que no sirva reproducir una grabación. Exige relojes en hora y se puede desactivar en Ajustes. No se aplica a los WAV.

Un receptor con clave rechaza mensajes sin cifrar, y uno sin clave no acepta los cifrados. La clave no sale del navegador.

La web. Las cabeceras de `vercel.json` solo permiten cargar recursos del propio sitio, impiden incrustar la página en otra y limitan el micrófono al propio sitio. Los WAV se validan, con un máximo de 100 MB.

> Los bloques criptográficos son estándar, pero la construcción es propia y ningún tercero la ha auditado: no debe usarse para información sensible. Además, cualquier micrófono cercano recibe la señal.

6. La interfaz

- **Mensajes.** Historial, línea de estado y caja de envío. El estado muestra «Escuchando», «Recibiendo… 40 %» o «Enviando», con barra de avance.
- **Espectro.** Un `AnalyserNode` entrega la potencia por frecuencia, y la página dibuja el espectro y una cascada de los últimos segundos. No interviene en la decodificación.
- **Ajustes.** Volumen, protocolo, repeticiones, clave, WAV y una prueba sin micrófono que añade ruido a la señal y la decodifica en la misma página.
- **Pantalla.** Mientras escucha, pide mantenerla encendida con Wake Lock, porque en el móvil apagarla puede cortar el micrófono. Avisa si la página pasa a segundo plano.
- **Actualizaciones.** Los archivos llevan una versión en la dirección (`?v=`), que se cambia en cada publicación para que nadie se quede con una copia antigua.

7. Valores principales

Parámetro	Valor	Para qué
Frecuencia de muestreo	48 000 Hz	Base de todos los tiempos
Trama y FFT	1024 muestras	21,3 ms; bins de 46,875 Hz
Chirp	7200 muestras	150 ms, de 1,5 a 6 kHz
Guarda alrededor de los datos	384 muestras	8 ms de silencio
Amplitud de cada tono	0,36	Dos tonos sin saturar
Umbrales de chirp	0,07 y 0,25	Inicio y fin

Parámetro	Valor	Para qué
Desplazamientos de ventana	0, −256, −512, +256	Búsqueda de alineación
Trozo de texto / bloque de longitud	48 B / 9 B	Fragmentación y longitud
Copias sin chirp final / abandono / tope	3 / 30 s / 90 s	Límites de la recepción

8. Cómo se comprueba

- **Pruebas automáticas.** 71 en Python (`python -m pytest`) y 84 en JavaScript (`web/test/roundtrip.mjs` y `stream.mjs`). Incluyen WAV generados por Python y decodificados en JavaScript, salas simuladas, falsas detecciones y entradas inválidas.
- **Por aire.** `diagnostico_aire/grabar_serie.py` graba capturas reales, y `comparar_receptores.mjs` las pasa por dos versiones del receptor sobre el mismo audio.
- **En el navegador.** Se sustituye el micrófono por una señal conocida, lo que prueba todo el camino, con los tres hilos.
- **Publicación.** Se lanza a mano con `npx vercel deploy --prod` desde `SpacesWases/web`; un push a GitHub no publica nada.

9. Límites del diseño

- **Velocidad.** Con 1 byte por símbolo la tasa es de 17 a 33 bps con 16 bytes; ggwave envía 3 bytes por símbolo.
- **Sentido único.** Un dispositivo no recibe mientras emite, y dos que emiten a la vez se interfieren.
- **Sin adaptación al canal.** No hay ecualización ni selección de tonos, así que los huecos de frecuencia de un altavoz se compensan solo con símbolos largos y paridad.
- **Bloque de longitud dañado.** Sin repeticiones, el mensaje no se puede decodificar.
- **Validación.** Las mediciones por aire usan un portátil y una sala, y los umbrales se eligieron con ellas. Falta una serie medida entre dos dispositivos reales.